INF 345 — Fundamentals of DevOps

Lecture 8: Playbooks, Roles & Variables

Adil Akhmetov · Lesson 8

Recap — Lesson 7 (Ansible Basics)

- `ansible all -m ping` fails with an SSH error for `localhost`. Why? (no `ansible_connection=local` — Ansible uses SSH for every host unless told otherwise)
- The second run of a playbook still says `changed=1` on the same task. Why? (a `shell` / `command` task — Ansible can't check its state, so it always reports "changed")
- When does a handler run? (once, at the end of the play, and only if a task that notifies it reported `changed`)

Today's agenda

- ☐ Playbooks in depth: several plays, `become`, tags
- ☐ Variables: every place they can live — and which one wins
- ☐ Facts, registered results, `when:` and loops
- ☐ Jinja2 templates and filters; handlers, revisited
- ☐ Roles: the standard way to package automation
- ☐ Worked example: last week's `site.yml` → a `webapp` role
- ☐ Straight into today's practice: refactor into a role

The scenario

Last week's `site.yml` worked. Now staging needs port 8081, production port 80, and one server is on Rocky Linux instead of Ubuntu. A teammate wants the same nginx setup for another project, so they copy your playbook and start editing it. Three months later there are four slightly different copies, and nobody knows which one is right.

Today: one piece of automation, reused everywhere, with the differences kept in variables instead of in copies.

Block 1

Playbooks in depth

A playbook is a list of plays

```yaml
- name: Databases
  hosts: db
  become: true
  tasks:
    - name: PostgreSQL installed
      ansible.builtin.package:
        name: postgresql
        state: present

- name: Web servers
  hosts: web
  become: true
  tasks:
    - name: nginx installed
      ansible.builtin.package:
        name: nginx
        state: present

- name: Smoke test
  hosts: localhost
  tasks:
    - name: Site answers
      ansible.builtin.uri:
        url: http://web1.example.com/
```

- A **play** = which hosts + how (`become` , vars) + what (tasks, roles, handlers).
- Plays run **top to bottom**. Database first, then web, then a check.
- Inside a play, each task runs on **all** its hosts before the next task starts.
- One file, several groups of machines, one command: `ansible-playbook site.yml` .

`become` : who runs the task

```yaml
- name: Web servers
  hosts: web
  become: true                  # every task in this play runs with sudo

  tasks:
    - name: nginx installed     # needs root: inherits become: true
      ansible.builtin.package:
        name: nginx
        state: present

    - name: Check the app as a normal user
      ansible.builtin.command: whoami
      become: false             # task-level override wins over play-level
      changed_when: false

    - name: Create the app database
      community.postgresql.postgresql_db:
        name: inf345
      become_user: postgres     # sudo to another user, not root
```

`become` can be set on a play, a block, a task, or a role. If sudo asks for a password, run with `-K` (`--ask-become-pass`).

Tags: run only part of a playbook

```yaml
- name: nginx installed
  ansible.builtin.package:
    name: nginx
    state: present
  tags: packages

- name: Site config
  ansible.builtin.template:
    src: app.conf.j2
    dest: /etc/nginx/conf.d/app.conf
  tags: config
```

```
ansible-playbook site.yml --list-tags
ansible-playbook site.yml --tags config
ansible-playbook site.yml --skip-tags packages
```

- "I only changed the config — don't reinstall everything."
- Tasks without the tag are skipped. The special tag `always` runs even with `--tags`.
- Use them sparingly: a playbook must still work when run **in full**.

Block 2

Variables: where they live, who wins

Where a variable can live

Where	Scope	Use it for
`roles/x/defaults/main.yml`	the role	defaults anyone can override
inventory `[web:vars]`	a group	quick tests — prefer `group_vars/`
`group_vars/web.yml`	hosts in `web`	per-group settings
`host_vars/web1.yml`	one host	the one machine that’s different
play `vars:` / `vars_files:`	one play	values tied to this playbook
`roles/x/vars/main.yml`	the role	internal constants
`-e` / `--extra-vars`	everything	one-off overrides

`group_vars/` and `host_vars/`

```
project/
├── ansible.cfg
├── inventory.ini
├── site.yml
├── group_vars/
│   ├── all.yml          # every host
│   └── web.yml          # hosts in [web]
└── host_vars/
    └── web2.example.com.yml
```

```yaml
# group_vars/web.yml
app_port: 80
app_env: production
```

- Ansible loads these **automatically**: no `include`, no `vars_files`.
- They must sit **next to the inventory or next to the playbook**.
- File name = **group name** or **host name** exactly (`web.yml`, a directory `web/` works too).
- Plain YAML, no `---` needed. Keep the inventory file a list of hosts, and keep the values here.
- `all.yml` is the weakest group; a specific group (`web`) beats `all`.

Precedence: who wins

From **weakest** (1) to **strongest** (8), simplified:

1. **Role defaults** — `defaults/main.yml`
2. Inventory group vars: `[web:vars]`, then `group_vars/all`, then `group_vars/web`
3. Inventory host vars, then `host_vars/<host>`
4. Facts gathered from the host
5. Play `vars:`, then `vars_files:`
6. **Role vars** — `vars/main.yml`
7. Task vars, `set_fact`, `register`
8. **Extra vars** — `-e` on the command line

- **Extra vars always win.** Nothing in a file can override `-e`.
- **Role defaults are the weakest.** Anything, anywhere, overrides them. That's exactly what you want for a tunable value.
- More specific beats more general: host beats group, `web` beats `all`.
- Role **vars** beat inventory — so `group_vars/` can't override them.
- The full list has 22 levels (in the Ansible docs). In practice: put each variable in **one** place.

Precedence in action

```
# roles/webapp/defaults/main.yml
app_port: 8080
```

```
# group_vars/web.yml          (web1 and web2 are in [web])
app_port: 80
```

```
# host_vars/web2.example.com.yml
app_port: 8081
```

- `ansible-playbook site.yml` → web1 listens on **80**, web2 on **8081**. The default 8080 is used by nobody — both hosts override it.
- A new host `web3` with no host_vars → **80** (from the group).
- A host outside `web` that uses the role → **8080** (the default).
- `ansible-playbook site.yml -e app_port=9090` → **every host: 9090**.

Quick check

You put `app_port: 8080` in `roles/webapp/vars/main.yml` and `app_port: 80` in `group_vars/web.yml`. Which port do the web servers get?

8080. Role `vars/` beat inventory variables. Move it to `defaults/main.yml` and the `group_vars` value (80) wins.

Block 3

Facts, conditionals, loops

Facts: what Ansible learns about each host

```yaml
- name: Show what Ansible found
  ansible.builtin.debug:
    msg: >-
      {{ inventory_hostname }} runs
      {{ ansible_facts['distribution'] }}
      {{ ansible_facts['distribution_version'] }}
      ({{ ansible_facts['os_family'] }} family)
```

```
ok: [web1] => {
    "msg": "web1 runs Ubuntu 24.04 (Debian family)"
}
```

```
ansible web1 -m ansible.builtin.setup     # all facts
```

- The **Gathering Facts** step at the start of every play runs the `setup` module and stores the results in `ansible_facts`.
- Useful ones: `os_family` (`Debian`, `RedHat`), `distribution`, `distribution_major_version`, `hostname`, `default_ipv4.address`, `memtotal_mb`, `processor_vcpus`.
- `inventory_hostname` is not a fact — it's the name from your inventory.
- Not using facts? `gather_facts: false` makes the play start faster.

`when:` — run a task only if…

```yaml
- name: nginx on Debian / Ubuntu
  ansible.builtin.apt:
    name: nginx
    state: present
  when: ansible_facts['os_family'] == 'Debian'

- name: nginx on RHEL / Rocky
  ansible.builtin.dnf:
    name: nginx
    state: present
  when: ansible_facts['os_family'] == 'RedHat'
```

- On an Ubuntu host the second task says `skipping: [web1]`.
- `when:` is **already** a Jinja2 expression — write the condition with no braces: `when: app_env == 'production'`, not `when: "{{ ... }}"`.
- A list under `when:` means **and**: every line must be true.

`register:` — keep a task's result

```yaml
- name: Is there a maintenance flag?
  ansible.builtin.stat:
    path: /etc/maintenance
  register: maint

- name: Look at the result
  ansible.builtin.debug:
    var: maint.stat.exists

- name: Deploy the page
  ansible.builtin.copy:
    src: index.html
    dest: /var/www/inf345/index.html
    mode: "0644"
  when: not maint.stat.exists
```

- `register: maint` saves the module's JSON result in a variable for this host — the same JSON you see with `-v`.
- Every module returns different fields: `stat` gives `maint.stat.exists`, `command` gives `.stdout`, `.rc`; every result has `.changed`.
- Not sure what's inside? `debug: var: maint` and read it.
- A read-only `command` you only register? Add `changed_when: false` so it doesn't report "changed" every run.

Loops: one task, many items

```yaml
- name: App directories
  ansible.builtin.file:
    path: "{{ app_root }}/{{ item }}"
    state: directory
    mode: "0755"
  loop:
    - static
    - logs
    - uploads

- name: App users
  ansible.builtin.user:
    name: "{{ item.name }}"
    groups: "{{ item.groups }}"
    append: true
  loop: "{{ app_users }}"

# group_vars/web.yml
# app_users:
#   - { name: deploy, groups: www-data }
#   - { name: ci,     groups: www-data }
```

- `loop:` takes a list; each element is `item` inside the task.
- Items can be dictionaries: `item.name`, `item.groups`.
- The list can come from a variable — so `group_vars` decides *how many*.
- Output shows each item: `changed: [web1] => (item=static)`.
- `when:` on a looped task is checked **per item**.
- Packages: pass the list to `name:` instead of looping — one install, much faster.

Quick check

A play has one task with `when: ansible_facts['os_family'] == 'RedHat'`, and the play has `gather_facts: false`. What happens?

It **fails**: `ansible_facts` was never collected, so `os_family` is undefined. Conditions on facts need fact gathering on.

Block 4

Templates, filters, handlers

Jinja2 templates: more than `{{ var }}`

`templates/app.conf.j2`

```
# {{ app_name | upper }} ({{ app_env }}) — managed by Ansi
server {
    listen {{ app_port }};
    server_name {{ server_names | join(' ') }};
    root {{ app_root }};
{% if app_env == 'production' %}
    server_tokens off;
{% endif %}
{% for path in blocked_paths %}
    location {{ path }} { deny all; }
{% endfor %}
}
```

- `{{ ... }}` prints a value. `{% ... %}` is logic: `if`, `for`.
- `{# ... #}` is a comment that never reaches the server.
- The same variables as everywhere else: role defaults, group_vars, facts, `-e`.
- Rendered **on the control node**, then copied — the server never needs Jinja2.
- A header like "managed by Ansible" warns people not to edit it by hand.

Filters: transform a value with `|`

```
{{ app_env | default('development') }}        {# app_env not set   → development   #}
{{ app_name | upper }}                        {# app_name: inf345  → INF345        #}
{{ server_names | join(', ') }}               {# [a.kz, b.kz]      → a.kz, b.kz    #}
{{ server_names | length }}                   {# [a.kz, b.kz]      → 2             #}
{{ app_port | int + 1 }}                      {# app_port: "8080"  → 8081          #}
```

- Filters chain left to right: `{{ ['a', 'b'] | join('-') | upper }}` → `A-B`.
- Without `default`, an undefined variable **fails the task**: `'app_workers' is undefined`.
- In a role, prefer a real default in `defaults/main.yml` over `| default(...)` scattered through templates — one place to look.

Handlers, revisited

```yaml
tasks:
  - name: Site config
    ansible.builtin.template:
      src: app.conf.j2
      dest: /etc/nginx/conf.d/app.conf
    notify: Reload nginx

  - name: TLS certificate
    ansible.builtin.copy:
      src: app.crt
      dest: /etc/nginx/app.crt
    notify: Reload nginx

handlers:
  - name: Reload nginx
    ansible.builtin.service:
      name: nginx
      state: reloaded
```

- Both tasks changed → the handler runs **once**, not twice.
- Neither changed (`ok`) → the handler **doesn't run at all**.
- It runs after the play's tasks finish, and `notify:` matches the handler's `name` exactly.
- If a later task fails on that host, its pending handlers are skipped. Need it right now? `ansible.builtin.meta: flush_handlers` .

```
RUNNING HANDLER [Reload nginx] ****
changed: [web2]       ← only web2's config changed
```

Block 5

Roles

Why roles

- A 200-line `site.yml` mixes **what** (a web app) with **where** (these hosts, these ports). You can't reuse half of it.
- A **role** packages one job — "set up the web app" — as a directory with a fixed layout: tasks, handlers, templates, files, defaults.
- The playbook shrinks to *which hosts get which roles*.
- Every host or environment changes the role's behaviour through **variables**, never by editing the role.
- Same idea as a function: a role is the body, its defaults are the parameters with default values.
- Shared roles and collections live on **Ansible Galaxy** — RH294 shows you how to install them.

The role layout

```
$ ansible-galaxy role init --init-path roles webapp
- Role webapp was created successfully

roles/webapp/
├── defaults/main.yml   # default values — weakest
├── files/              # static files for copy
├── handlers/main.yml   # handlers
├── meta/main.yml       # author, platforms, dependencies
├── tasks/main.yml      # the entry point: task list
├── templates/          # .j2 files for template
├── tests/              # sample inventory + playbook
├── vars/main.yml       # internal constants — strong
└── README.md
```

- `ansible-galaxy init webapp` (short form) creates the same thing in the current directory.
- Ansible looks for `main.yml` in each folder — that's the role's entry point.
- Delete the folders you don't use. Only `tasks/` is really needed.
- Inside the role, `src:` paths are **relative to the role**: `copy` looks in `files/`, `template` looks in `templates/`.
- Ansible finds roles in a `roles/` directory **next to the playbook**.

`defaults/` vs `vars/`

`defaults/main.yml` — the role's settings

```yaml
app_name: inf345
app_port: 8080
app_env: production
```

Weakest of all: group_vars, host_vars, play vars and `-e` override them. This is the role's public interface.

`vars/main.yml` — the role's internals

```yaml
nginx_conf_dir: /etc/nginx/conf.d
```

Stronger than inventory: group_vars **can't** override them. Use it for values the role relies on and users shouldn't touch.

Rule of thumb: if someone might want to change it, it goes in `defaults/`. When in doubt — `defaults/`.

Using a role in a play

```yaml
- name: Web servers
  hosts: web
  become: true
  roles:
    - webapp

- name: A second site, same role
  hosts: web
  become: true
  roles:
    - role: webapp
      vars:
        app_name: docs
        app_port: 8090
```

- `roles:` runs the role's `tasks/main.yml`, with its handlers, defaults and vars loaded.
- Pass values per use with `vars:` under the role (they beat `group_vars`).
- Order inside a play: `roles:` run **before** `tasks:`. You can have both.
- Need a role in the middle of a task list? `ansible.builtin.import_role` or `include_role` as a task.

Worked example

Take last week's `site.yml` — nginx serving one page, a templated config, a handler — and turn it into a role called `webapp`. Same result on the servers, but reusable and configurable.

Before: everything in one file

```yaml
- name: Web servers
  hosts: web
  become: true
  vars:
    app_name: inf345
    app_port: 8080
    app_root: /var/www/inf345
  tasks:
    - { name: nginx installed, ansible.builtin.package: { name: nginx, state: present } }
    - name: App directory
      ansible.builtin.file: { path: "{{ app_root }}", state: directory, mode: "0755" }
    - name: Home page
      ansible.builtin.copy: { src: files/index.html, dest: "{{ app_root }}/index.html", mode: "0644" }
    - name: Site config
      ansible.builtin.template:
        src: templates/app.conf.j2
        dest: /etc/nginx/conf.d/app.conf
        mode: "0644"
      notify: Reload nginx
  handlers:
    - name: Reload nginx
      ansible.builtin.service: { name: nginx, state: reloaded }
```

Values, steps and handlers are all tangled together. Four parts → four role folders.

After (1/3): defaults and tasks

`roles/webapp/defaults/main.yml`

```yaml
---
app_name: inf345
app_port: 8080
app_env: production
app_root: /var/www/inf345
server_names:
  - localhost
```

The `vars:` block moved here — now they're **defaults**, not fixed values.

`roles/webapp/tasks/main.yml`

```yaml
---
- name: nginx installed
  ansible.builtin.package:
    name: nginx
    state: present

- name: App directory
  ansible.builtin.file:
    path: "{{ app_root }}"
    state: directory
    mode: "0755"

- name: Home page
  ansible.builtin.copy:
    src: index.html        # → files/index.html
    dest: "{{ app_root }}/index.html"
    mode: "0644"
```

After (2/3): template, config task, handler

`roles/webapp/templates/app.conf.j2`

```
# {{ app_name | upper }} — managed by Ansible
server {
    listen {{ app_port }};
    server_name {{ server_names | join(' ') }};
    root {{ app_root }};
}
```

`roles/webapp/handlers/main.yml`

```yaml
---
- name: Reload nginx
  ansible.builtin.service:
    name: nginx
    state: reloaded
```

`roles/webapp/tasks/main.yml` (continued)

```yaml
- name: Site config
  ansible.builtin.template:
    src: app.conf.j2       # → templates/app.conf.j2
    dest: "/etc/nginx/conf.d/{{ app_name }}.conf"
    mode: "0644"
  notify: Reload nginx

- name: nginx running
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: true
```

No `files/` or `templates/` prefix in `src:` — the role knows where to look.

After (3/3): the playbook and the override

`site.yml` — now 5 lines

```yaml
- name: Web servers
  hosts: web
  become: true
  roles:
    - webapp
```

`group_vars/web.yml` — this group's differences

```yaml
app_port: 80
server_names:
  - inf345.example.kz
  - www.inf345.example.kz
```

```
ansible-playbook site.yml           # listen 80; both name
ansible-playbook site.yml           # changed=0, no reloa
ansible-playbook site.yml -e app_port=9090   # one change
```

- The role wasn't edited — `group_vars` overrode two **defaults**.
- Second run: nothing changed, so the handler didn't run.
- Task names in the output now carry the role: `TASK [webapp : Site config]`.

Quick check

You ran the playbook, then edited only `files/index.html` in the role and ran it again. Does nginx reload?

No. The "Home page" task changed, but it doesn’t notify the handler — only "Site config" does, and the config didn’t change. (nginx serves the new page anyway: no reload needed for static files.)

Block 6

Common mistakes

When it doesn’t work — variables

Symptom	Likely cause
`group_vars` value ignored	It’s set in the role’s `vars/` (or with `-e`) — move it to `defaults/`
`group_vars/web.yml` never loaded	Not next to the inventory or playbook, or the file name isn’t the group name
YAML error on a line starting with `{{`	Quote it: `path: "{{ app_root }}"`
Warning about templating in `when:`	`when:` takes a bare expression — drop the `{{ }}`

When it doesn’t work — roles and handlers

Symptom	Likely cause
`the role 'webapp' was not found`	No `roles/webapp/` next to `site.yml`
Template or file "not found" in a role	`src:` is relative to the role: `app.conf.j2`, not `templates/app.conf.j2`
`'app_port' is undefined`	Missing from `defaults/main.yml`
Handler never runs	Nothing changed, the `notify:` name doesn’t match, or a later task failed

Self-check

- `defaults/` says `app_env: production`, `group_vars/web.yml` says `staging`, you run with `-e app_env=test`. Result? (`test` — extra vars always win)
- Your play notifies `Reload nginx` from three tasks; all three changed. How many reloads? (one, after the tasks finish)
- What does `{{ ['web1', 'web2'] | join(', ') | upper }}` print? (`WEB1, WEB2`)
- Where in a role do you put a value users of the role should be able to change? (`defaults/main.yml`)

Block 7

Today’s practice

Practice 08 — Refactor into a role

Last week's playbook, split into a role. The role skeleton, template and page are given; you fill in:

1. `site.yml` — one play on `web`, just `roles: [webapp]`
2. `defaults/main.yml` — last week's values, as defaults
3. `tasks/main.yml` — last week's tasks, moved into the role
4. the template task **notifies** a handler
5. `handlers/main.yml` — the handler: runs only on change
6. `group_vars/web.yml` — override one default

```
ansible-playbook site.yml --syntax-check
ansible-playbook site.yml      # works, handler runs
ansible-playbook site.yml      # changed=0, no handler
ansible-playbook site.yml -e app_port=9090
```

Same flow as last week: accept "Practice 08" on Maru, work in your repo, push to `main`. Graded automatically on every push — the role layout, the default and its override, the template, the handler, and idempotence.

By the end of this lesson, you should be able to

- ☐ Write a playbook with several plays, `become` and tags
- ☐ Put a variable in the right place — and predict which value wins
- ☐ Use facts, `register` and `when:` to make tasks conditional
- ☐ Loop over a list, and use filters in a Jinja2 template
- ☐ Explain when a handler runs — and when it doesn't
- ☐ Turn a playbook into a role with defaults, a template and a handler

Before next lecture

- ☐ Finish today’s practice if you didn’t wrap it up in session — same Maru flow as always
- ☐ **Lab 02 (RHA RH294)** is due **Wed, Nov 4, 23:59** — the chapters on variables and facts, task control, templates and roles are today’s material; see "Red Hat labs" in Maru
- ☐ Two weeks left on Lab 02: don’t leave the whole course for the last weekend

Keep today's role — next week we test it.

Next lecture

Idempotence & testing with Molecule — proving a role works (and keeps working) before it ever touches a real server.